



RIVER VALLEY HIGH SCHOOL
General Certificate of Education Advanced Level
Higher 2
J2 Prelim

COMPUTING

Paper 2 (Lab-based)

9569/02

19 Aug 2026

3 hours

Additional Materials: Electronic version of the following data file
in the folder named “resource folder”

- `strikers.txt`
- `useful_code.txt`
- `got_talent.db`

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

All your solutions must be placed in the folder named:

`<thumb_drive_number>_<class>_<name>_output` folder for submission.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to 6 marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each question or part question. The total number of marks for this paper is 100.

This document consists of 19 printed pages and 1 blank page.

1 Snakes and Ladders

Name your Jupyter Notebook as

Task_1_<class>_<index number>_<name>.ipynb

For each of the Task 1.1 to 1.7, complete the codes in Jupyter Notebook and add a comment statement at the beginning of the code using the hash symbol #, to indicate the sub-task the program code belongs to, for example

In [1]: `#Task 1.1
Program code`

Output:

A simplified **Snakes and Ladders** game is represented using a singly linked list of tiles. Each tile has a unique tile ID and is connected to the next tile on the game board. Some tiles contain ladders that allow a player to climb to another tile, while others contain snakes that cause a player to slide to an earlier tile.

In this task, you will implement the classes and functions required to create the game board, traverse the linked list, locate specific types of tiles, move along the path, and activate the snakes and ladders according to the given configuration.

Task 1.1

Implement the classes `Tile`, `Ladder` and `Snake` based on the information described below.

[8]

Class <code>Tile</code>	
Attributes	
<code>tile_id</code>	<integer> It stores the ID of the tile.
<code>next_tile</code>	<Tile> It stores a reference to the next tile in the path. It is set to <code>None</code> if there is no next tile.
Functions	
getters and setters	The getter and setter functions for all the attributes of the class <code>Tile</code> .
<code>Tile(tile_id)</code>	The constructor of the class, the ID of the tile is set to <code>tile_id</code> and the attribute <code>next_tile</code> is set to <code>None</code> .
<code>__str__()</code>	This function returns a string in the following format: "N03" where N means normal tile and the tile ID should always be displayed using two digits. <i>There will not be more than 99 tiles.</i>

Class Ladder (inherits from Tile)	
Attributes	
ladder_tile	<Tile> It stores the destination tile reached after climbing the ladder. This is assigned to <code>None</code> when a new <code>Ladder</code> Tile is first created.
Functions	
getters and setters	The getter and setter functions for all the attributes of the class <code>Ladder</code> .
<code>Ladder(tile_id)</code>	The constructor of the class, the ID of the tile is set to <code>tile_id</code> and the attribute <code>ladder_tile</code> is set to <code>None</code> .
<code>__str__()</code>	If the <code>ladder_tile</code> has not been assigned, this function returns a string in the format: <code>"L05"</code> where <code>L</code> means ladder tile and <code>05</code> is its tile ID. Otherwise, it returns: <code>"L05 goes to 08"</code> where <code>08</code> is the destination tile ID in this example.

Class Snake (inherits from Tile)	
Attributes	
snake_tile	<Tile> It stores the destination tile reached after sliding down the snake. This is assigned to <code>None</code> when a new <code>Snake</code> Tile is first created.
Functions	
getters and setters	The getter and setter functions for all the attributes of the class <code>Snake</code> .
<code>Snake(tile_id)</code>	The constructor of the class, the ID of the tile is set to <code>tile_id</code> and the attribute <code>snake_tile</code> is set to <code>None</code> .
<code>__str__()</code>	If the <code>snake_tile</code> has not been assigned, this function returns a string in the format: <code>"S08"</code> where <code>S</code> means snake tile and <code>08</code> is its tile ID. Otherwise, it returns: <code>"S08 goes to 12"</code> where <code>12</code> is the destination tile ID in this example.

Task 1.2

Write a function `create_path(path_str)` that takes a string `path_str` as parameter and returns the first tile of a linked path (a singly linked list).

The first character of `path_str` is always "N" and represents the starting normal tile with tile ID 0.

For each remaining character:

"N" represents a normal Tile.

"L" represents a Ladder Tile.

"S" represents a Snake Tile.

The tile ID of each tile is equal to its position in the string.

All tiles should be connected using their `next_tile` references.

For example,

```
>>> path = create_path("NLNS")
>>> print(path)
>>> print(path.next_tile)
>>> print(path.next_tile.next_tile.next_tile)
N00
L01
S03
```

creates the following path (a singly linked list).

N00 → L01 → N02 → S03

The function returns the first tile (N00).

[4]

Task 1.3

Write a function `display_tiles(path)` that takes the first tile path of a linked path as input and displays every tile from the beginning to the end using the `__str__()` function of each tile.

Test your function using `test1_3()`.

```
def test1_3():
    path = create_path("NLNNNSNNNNLNNN")
    display_tiles(path)
```

[3]

Task 1.4

Write the following two functions.

- `find_next_ladder(tile)`

This function takes a `Tile` instance `tile` as input and returns the first Ladder encountered after `tile`. If no ladder exists after `tile`, return `None`.

- `find_next_snake(tile)`

This function takes a `Tile` instance `tile` as input and returns the first Snake encountered after `tile`. If no snake exists after `tile`, return `None`.

Test your function using `test1_4()`.

```
def test1_4():

    path = create_path("NLNNNNSSNNNLNNN")
    curr = path

    curr = find_next_ladder(curr)
    print(curr) # This should output L01

    curr = find_next_snake(curr)
    print(curr) # This should output S06

    curr = find_next_ladder(curr)
    print(curr) # This should output S11
```

[5]

Task 1.5

Write a function `move_forward(tile, n)` that takes a `Tile` instance `tile` and an integer `n` as parameters.

The function moves `n` tiles forward along the linked path and returns the destination tile.

If the end of the path is reached before completing all `n` steps, the function returns the last tile instead.

Test your function using `test1_5()`.

```
def test1_5():

    path = create_path("NLNNNNSNNNNLNNN")
    curr = path

    curr = move_forward(curr, 2)
    print(curr) # This should output N02

    curr = move_forward(curr, 6)
    print(curr) # This should output N08

    curr = move_forward(curr, 20)
    print(curr) # This should output N14 (the last tile) [4]
```

Task 1.6

Write a function `activate_ladder_tiles(path, ladder_str)` that takes the first tile `path` and a string `ladder_str` as parameters.

The string is in the following format:

`L, destination1, destination2, ...`

Each destination corresponds to the next ladder found in the linked path. The first ladder is connected to the first destination tile, the second ladder to the second destination tile, and so on.

For example,

`path: N00 → N01 → L02 → N03 → N04 → L05 → N06`

`activate_ladder_tiles(path, "L,4,6")` results in

`L02 goes to 04`

`L05 goes to 06`

You can assume that the number of destination tiles specified in `ladder_str` is always the same as the number of ladders in the path.

Test your function using `test1_6()`.

```
def test1_6():
    path = create_path("NNLNNNSNNNLNNSNN")
    activate_ladder_tiles(path, "L,3,13")
    display_tiles(path)
```

The following should be displayed.

```
N00
N01
L02 goes to 03
N03
N04
S05
N06
N07
N08
L09 goes to 13
N10
N11
S12
N13
N14
```

[6]

Task 1.7

Write a function `activate_snake(path, snake_str)` that takes the first tile path and a string `snake_str` as parameters.

The string is in the following format:
`S, destination1, destination2, ...`

Each destination corresponds to the next snake found in the linked path. For each snake, set its destination tile according to the corresponding destination tile ID specified in `snake_str`.

For example:

path: `N00 → S01 → N02 → N03 → S04 → N05 → N06`

```
activate_snake(path, "S,0,2")
```

results in:

`S01 goes to 00`

`S04 goes to 02`

You can assume that the number of destination tiles specified in `snake_str` is always the same as the number of snakes in the path.

Test your function using `test1_7()`.

```
def test1_7():
    path = create_path("NNLNNSNLNLNNSNN")
    activate_snake(path, "S,3,6")
    display_tiles(path)
```

The following should be displayed.

```
N00
N01
L02
N03
N04
S05 goes to 03
N06
L07
N08
L09
N10
N11
S12 goes to 06
N13
N14
```

[8]

2 School Soccer

Name your Jupyter Notebook as

Task_2_<class>_<index number>_<name>.ipynb

For each of the Task 2.1 to 2.4, complete the codes in Jupyter Notebook and add a comment statement at the beginning of the code using the hash symbol #, to indicate the sub-task the program code belongs to, for example

In [1]: `#Task 2.1
Program code`

Output:

In this task, you will work with a dataset containing statistics of soccer strikers. You will write Python functions to read, search, filter, sort and analyse the data to obtain useful information about the players and their schools.

The data is stored in the text file “*strikers.txt*”. It contains the season statistics of 50 strikers in **12** matches. Each row represents one striker and contains information about the school they represent, match results, goal attempts, goals, assists and cards received. The first row contains the field names.

Field	Data type	Description
Name	<string>	The name of the striker.
School	<string>	The school represented by the striker.
Wins	<integer>	Number of matches won.
Draws	<integer>	Number of matches drawn.
Losses	<integer>	Number of matches lost.
Goal Attempts	<integer>	Number of goal attempts.
Goals	<integer>	Number of goals scored.
Goals Assisted	<integer>	Number of goals assisted.
Yellow Cards	<integer>	Number of yellow cards received.
Red Cards	<integer>	Number of red cards received.

Task 2.1

Write a function `task2_1(filename)` that takes the filename of the dataset as a parameter.

The function should read the file, skip the header row, convert all numerical fields to integers, and store each record as a list in a 2D list.

The function should return the 2D list.

You may assume that the file exists and that all records follow the specified format.

Test your function and show the output using:

```
players = task2_1("strikers.txt")

for player in players:
    print(player)
```

[3]

Task 2.2

Write a function `task2_2(strikers)` with `strikers` as a parameter. The parameter contains the 2D list returned by `task2_1("strikers.txt")`.

The function should return a list of names of strikers who have participated in **all 12 matches**.

Output the names in a list.

[3]

Task 2.3

Write a function `task2_3(strikers)` with `strikers` as a parameter. The parameter contains the 2D list returned by `task2_1("strikers.txt")`.

For each striker, calculate the sum of their **Goals** and **Goals Assisted**.

The function should return a 2D list containing the **name** and **combined total** of every striker who has the highest combined total.

If more than one striker has the highest combined total, all such strikers should be included in the returned list.

The time complexity of this function must be in **$O(n)$** .

Output the names and its respective scores in a 2D list.

[4]

Task 2.4

Write a function `task2_4(strikers)` with `strikers` as a parameter. The parameter contains the 2D list returned by `task2_1("strikers.txt")`.

The function should output the **Average Goals** of each school in descending order in the format as shown below.

School	Average Goals
-----	-----
Northview Academy	21.00
Cedar Grove High	19.25
Oakridge Secondary	18.00
.	
.	
.	
.	
.	
Eastwood High	11.25
Pinecrest High	10.33

You must perform **merge sort** to achieve the descending order.

[8]

3 25 unique Letters

Name your Jupyter Notebook as

Task_3_<class>_<index number>_<name>.ipynb

For each of the Task 3.1 to 3.3, complete the codes in Jupyter Notebook and add a comment statement at the beginning of the code using the hash symbol #, to indicate the sub-task the program code belongs to, for example

In [1]:

```
#Task 3.1
Program code
```

Output:

Task 3.1

Write a function `task3_1()` that asks the user to enter 25 unique letters, **in groups of five**.

For each input, the user should enter a string containing five letters. The function should continue asking the user to re-enter the string until a valid string is entered.

The following validation rules should be checked in the order given:

- If the string does not contain exactly five characters, display:

```
"You must type 5 letters each time."
```
- If any character is not a letter, display:

```
"You must only input letters. "
```
- If any letter is repeated within the current string, display:

```
"You must only input unique letters."
```
- If any letter has already been entered in a previous valid string, display:

```
"You have keyed in <these letters> before."
```


 where `<these letters>` should contain the letters that have already been entered.

Once a valid string has been entered, its five letters should be added to the 2D list.

The process continues until 25 unique letters have been entered. The function should return the letters as a 5×5 2D list.

For example:

```
>>> task3_1()
```

Key in 5 unique letters that you have not entered before:

ABCDE

Success!

Key in 5 unique letters that you have not entered before:

FFGHF

You must only input unique letters.

Try again: **FAAAG**

You must only input unique letters.

You have keyed in A before.

Try again: **HIJKLH**

You must type 5 letters each time.

You must only input unique letters.

Try again: **HIJKL**

Success!

Key in 5 unique letters that you have not entered before:

MNOP2

You must only input letters.

Try again: **MNOPL**

You have keyed in L before.

Try again: **MNOPQ**

Success!

Key in 5 unique letters that you have not entered before:

WXYZF

Success!

Key in 5 unique letters that you have not entered before:

RSTUV

Success!

```
['ABCDE', 'HIJKL', 'MNOPQ', 'WXYZF', 'RSTUV']
```

Show the output of your program using the input as shown in the example above.

[8]

Task 3.2

Write a function `task3_2(board, target_letter)` that takes the 5×5 2D list board created in the previous task and a character `target_letter` as parameters.

The board contains 25 unique letters.

The function should search through board and return the row and column position of `target_letter` as a tuple in the form `(row, column)`.

The row and column indices start from 0.

If `target_letter` cannot be found in board, the function should return `(-1, -1)`.

Test your function with the following codes.

```
print(task3_2(['ABCDE', 'HIJKL', 'MNOPQ', 'RSTUV', 'WXYZF'], 'A'))
print(task3_2(['ABCDE', 'HAJKL', 'MNOPQ', 'RSTUV', 'WXYZF'], 'J'))
print(task3_2(['RSTUV', 'IBCE', 'HLJKA', 'MNOPQ', 'WXYZF'], 'K'))
print(task3_2(['MNOPQ', 'WXYZF', 'RSTUV', 'HLAKJ', 'IBCE'], 'L'))
print(task3_2(['MNOPQ', 'WXYZF', 'RSTUV', 'IBCE', 'HLJKA'], 'H'))
print(task3_2(['MNOPQ', 'WXYZF', 'RSTUV', 'IBCE', 'HLJKZ'], 'A'))
```

[3]

Task 3.3

Write a function `task3_3(board)` that takes the 5×5 2D list board created in the previous task as a parameter. The board contains 25 unique letters.

The function should find the longest consecutive sequence of letters starting from **A**.

For each letter in the sequence, the next letter must be found in a tile that is immediately **adjacent horizontally or vertically** to the current letter. Diagonal positions are **not** considered adjacent.

The sequence must follow the alphabetical order.

For example:

$A \rightarrow B \rightarrow C \rightarrow D$ is a valid sequence if each letter is adjacent to the next letter.

The search should stop when the next letter in the sequence cannot be found in any of the four adjacent positions.

The function should return the number of letters in the sequence.

If A does not exist in the board, the function should return 0.

For example, if the board contains:

```
S V T U W
D C B A X
E F P R Y
Q G O N M
H I J K L
```

the sequence is:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Therefore,

```
>>> board = ['SVTUW', 'DCBAX', 'EFPRY', 'QGONM', 'HIJKL']
>>> task3_3(board)
7
```

Test your function with the following codes.

```
print(task3_3(['ABCDE', 'JIHGF', 'KLMNO', 'TSRQP', 'UVWXY']))
print(task3_3(['QPONM', 'REDCL', 'SFABK', 'TGHIJ', 'UVWXY']))
print(task3_3(['QRSTU', 'PEFGV', 'ODAHW', 'NCBIX', 'MLKJY']))
print(task3_3(['KLOPQ', 'JMNSR', 'IHGTU', 'DEFWV', 'CBAXY']))
print(task3_3(['KLOPQ', 'JMNSR', 'IHGYU', 'DEFWV', 'CBAXT']))
print(task3_3(['TVUWX', 'NPQRS', 'OABCY', 'KLMJI', 'EFGHD']))
print(task3_3(['MLKJI', 'NEFGH', 'ODYXW', 'PCBIV', 'QRSTA']))
print(task3_3(['MLKJI', 'NEFGH', 'ODYXW', 'PCBIV', 'QRSTU']))
```

[7]

4 Database Web Application

Name your Jupyter Notebook as

Task_4_<class>_<index number>_<name>.ipynb

For each of the Task 4.1 to 4.3, complete the codes in Jupyter Notebook and add a comment statement at the beginning of the code using the hash symbol #, to indicate the sub-task the program code belongs to, for example

In [1]: `#Task 4.1
Program code`

Output:

In River Valley Hypothetical High, you are tasked with designing and implementing a web-application with relational database system for the school's talent-time registration portal.

The following SQLite database `got_talent.db` has been created to store records of the different information and has 2 tables – `performance_types` and `participants`.

`performance_types`:

- `id` – id for the performance type; primary key. Example, P1.
- `description` – description of performance.
- `max_number` – maximum number of registrations for that performance type.

`participants`:

- `id` – identification of the participants; primary key.
- `name` – the participant's name.
- `performance` – the `id` of the performance type which the participant had registered for. This is a foreign key from table `performance_types`.

Task 4.1

The web application needs to retrieve participants who registered for a specific performance type `id`.

Write a python program using just 1 SQL statement to print out a `list` of names in tuples who registered for performance type of `id` P1.

For example:

```
[('Urban Crew',)]
```

[2]

Task 4.2

The web application needs to check for the number of vacancies for all performance type. `Vacancies` is calculated by taking the `max_number` of each performance type `id` and subtract away number of participants who registered for the performance type `id`.

Write a python program using just 1 SQL statement to print out a `list` of the performance type `id`, `description` and `vacancies` in tuples within a nested list.

For example:

```
[('P1', 'dance group', 1), ('P2', 'sing band', 0), ('P3', 'sing solo', 2), ('P4', 'stand up', 1)]
```

[4]

Task 4.3

The web application needs to register a participant into a specific performance type `id` after it is verified that the performance type is still available.

Write a python program using SQL statement(s) to register participant 'Oh Wee Wee' to performance type 'P1' into participants table while ensuring that the `id` is always unique even when the same program is used to register other participants into different performance type `id`.

For this task, there is no need to check for vacancies for the performance type. You are also not allowed to modify the schema for participants to include autoincrement.

Partial marks will be awarded for the use of autoincrement or hardcoding a unique participants `id`.

[4]

Task 4.4

Write a python program and the necessary folders and files to create a web application for the registration portal. Save all files and folders within a folder named `Task_4_4`.

The web application will display `home` page titled 'RV Got Talent'. The webpage will have the following:

- 1) a table of the performance type description and the vacancies left. If the vacancy number is filled up (ie the number of participants who signed reaches the `max_number` for the performance type), the content will display 'Closed'.
- 2) The vacancy number in the table is also a hyperlink to the `registration` page for the performance type `id`. No hyperlink for vacancies is 'Closed'.
- 3) Your webpage must be like the one shown in the next page.

Registration for RV Got Talent

S/N	Category	Vacancies
1	dance group	Closed
2	sing band	Closed
3	sing solo	<u>2</u>
4	stand up	<u>1</u>

Home page

The registration page will have the following:

- 1) a table showing the current participants who signed up for the performance type id.
- 2) a form for participant to enter his/her details which include
 - a. name text entry to enter his/her/their name/band name.
 - b. performance type id text entry with the performance type id pre-populated as the value. To make the text input not editable, add the attribute `readonly` to the text input tag.
E.g `<input type="text" name="abc" value="def" readonly>`
- 3) the form will be submitted to the home page once submitted
- 4) Your webpage must be similar to the one shown below.

Registration for sing solo

S/N	Name
1	Elena Rostova

Enter Name:

Performance Type:

registration page

On submission, the home page will be shown with refreshed vacancies.

Registration for RV Got Talent		
S/N	Category	Vacancies
1	dance group	Closed
2	sing band	Closed
3	sing solo	<u>1</u>
4	stand up	<u>1</u>

home page will be shown with refreshed vacancies

You are only allowed 2 html templates for this application.

Your web application should implement CSS to achieve similar look for as was shown. Some required CSS are as follows

- All content in the table is bold.
- 'Closed' text will be shown in red.
- Form width set to 500px
- Tables, form, page titles and submit button will be centered to the webpage.
- Assume your own values for margins and paddings for the required style.

Save your program code as

Task_4_4_<class>_<index>.py

With any additional files / subfolders as needed in a folder named

Task_4_4_<class>_<index>

Run the web application and save the returned pages in the root directory as

Task_4_4_1_<class>_<index>.html

Task_4_4_2_<class>_<index>.html

[10]

End of paper